C programming Day 7 Assignment

1. Student Record File

Write a C program to:

- Accept details of **5 students** (Roll No, Name, Marks).
- Store the records in a file named students.txt.
- Read the file and display all records.

2. Search a Word in a File

Write a C program to:

- Read a filename and a word from the user.
- Search whether the word exists in the file.
- Display the number of occurrences of the word.

3. Dynamic Array Using malloc()

Write a C program to:

- Read n integers using malloc().
- Display the entered numbers.
- Find the largest and smallest elements.

4. Student Marks Using calloc()

Write a C program to:

- Allocate memory for marks of n students using calloc().
- Input the marks.
- Calculate and display the average marks.

5. Even and Odd Count

Write a C program using calloc() to:

- Store n integers.
- Count and display the number of even and odd numbers.

6. Expand an Array

Write a C program to:

- Allocate memory for n integers using malloc().
- Read the elements.
- Increase the size of the array using realloc() to store m additional elements.
- Display the complete array.

7. Student List Update

Write a C program to:

- Initially allocate memory for details of 3 students.
- Later increase the memory using realloc() to add 2 more students.
- Display all student records.

8. Employee Information Using enum and union

Write a C program that:

- Defines an **enum** named EmployeeType with the values:
    - Manager

    - Engineer
    - Intern
- Defines a **union** named EmployeeData that can store either:
    - Monthly Salary (float), or
    - Stipend (float).
- Accept the employee type and the corresponding salary/stipend.
- Display the employee type and the stored value.

MCQs

Q1. Which file opening mode allows both reading and writing, creates the file if it does not exist, and truncates the file if it already exists?

A) "r+"

B) "w+"

C) "a+"

D) "rb+"

Answer: B) "w+"

Q2. Consider the following code:

```
FILE *fp = fopen("test.txt", "a");
fprintf(fp, "Hello");
fclose(fp);
```

If test.txt already contains ABC, the final content will be:

A) Hello

B) ABCHello

C) HelloABC

D) Runtime Error

Answer: B) ABCHello

Q3. What is the correct way to allocate memory for an array of 20 integers?

A) int *p = malloc(20);

B) int *p = malloc(20 * sizeof(int));

C) int *p = malloc(sizeof(p));

D) int *p = calloc(20);

Answer: B) int *p = malloc(20 * sizeof(int));

Q4. Which statement about calloc() is TRUE?

A) It allocates memory but does not initialize it.

B) It initializes allocated memory to zero.

C) It cannot allocate arrays.

D) It is faster than malloc() because it skips initialization.

Answer:B) It initializes allocated memory to zero.

Q5. Which statement about realloc() is correct?
A) It always allocates memory at the same address.
B) It may move the allocated block to a new location.
C) It can only increase memory size.
D) It automatically frees the original pointer even if allocation fails.
Answer: B) It may move the allocated block to a new location.

Q6. What happens if realloc() fails?
ptr = realloc(ptr, new_size);
A) Original memory is automatically freed.
B) ptr becomes NULL and original memory is lost.
C) Original memory remains allocated, but assigning directly to ptr may cause a memory leak.
D) Program terminates automatically.
Answer: C) Original memory remains allocated, but assigning directly to ptr may cause a memory leak.

Q7. Consider the code:
```
FILE *fp = fopen("abc.txt", "w");
fprintf(fp, "Programming");
fseek(fp, 3, SEEK_SET);
fprintf(fp, "XYZ");
fclose(fp);
```
What will be stored in the file?
A) ProgrammingXYZ
B) ProXYZmming
C) XYZgramming
D) Programming
Answer: B) ProXYZmming

Q8. Consider:
```
enum Day {MON = 2, TUE, WED, THU};
```
What is the value of THU?
A) 3
B) 4
C) 5
D) 6
Answer: C) 5

Explain

1. What is the difference between fputc() and fgetc()?
2. Explain the use of fputs() and fprintf().
3. Explain the syntax of malloc().
4. What does malloc() return on failure?
5. A programmer forgets to call free() after using malloc(). What problems may arise if this happens repeatedly?
6. How is calloc() different from malloc()?
7. Explain the syntax and working of calloc() with an example.
8. Can realloc() reduce the size of allocated memory? Explain.
9. Explain the syntax and advantages of enum with an example.
10. In what situations would you prefer using a union instead of a structure? Explain with an example.

Amit_Lab7 - Day 7 Assignment

C Programming - File Handling & Dynamic Memory Allocation

Question 1: Student Record File

```c
#include <stdio.h>

struct Student
{
    int rollNo;
    char name[50];
    float marks;
};

int main()
{
    struct Student s[5];
    int i;
    FILE *fp;

    fp = fopen("students.txt", "w");
    if(fp == NULL)
    {
        printf("Error opening file!\n");
        return 1;
    }

    for(i = 0; i < 5; i++)
    {
        printf("Enter details of student %d:\n", i + 1);

        printf("Roll No: ");
        scanf("%d", &s[i].rollNo);

        printf("Name: ");
        scanf("%s", s[i].name);

        printf("Marks: ");
        scanf("%f", &s[i].marks);
```

```
        fprintf(fp, "%d %s %.2f\n", s[i].rollNo, s[i].name, s[i].marks);
    }

    fclose(fp);

    fp = fopen("students.txt", "r");
    if(fp == NULL)
    {
        printf("Error opening file!\n");
        return 1;
    }

    printf("\n--- Records in students.txt ---\n");
    printf("%-10s %-20s %-10s\n", "Roll No", "Name", "Marks");

    {
        int rollNo;
        char name[50];
        float marks;

        while(fscanf(fp, "%d %s %f", &rollNo, name, &marks) != EOF)
        {
            printf("%-10d %-20s %-10.2f\n", rollNo, name, marks);
        }
    }

    fclose(fp);

    return 0;
}
```

Output:

```
PS C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C> ./test
Enter details of student 3:
Roll No: 3
Name: Akshansh
Marks: 90
Enter details of student 4:
Roll No: 4
Name: Golu
Marks: 96
Enter details of student 5:
Roll No: 5
Name: Shani
Marks: 99

--- Records in students.txt ---
Roll No    Name                Marks
1          Amit                99.00
2          Aryan               98.00
3          Akshansh            90.00
4          Golu                96.00
5          Shani               99.00
PS C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C>
```

Question 2: Search a Word in a File

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char filename[100], word[50], buffer[50];
    int count = 0;
    FILE *fp;

    printf("Enter filename: ");
    scanf("%s", filename);

    printf("Enter word to search: ");
    scanf("%s", word);

    fp = fopen(filename, "r");
    if(fp == NULL)
    {
        printf("Could not open file %s\n", filename);
```

```
        return 1;
    }

    while(fscanf(fp, "%s", buffer) != EOF)
    {
        if(strcmp(buffer, word) == 0)
        {
            count++;
        }
    }

    fclose(fp);

    if(count > 0)
    {
        printf("The word \"%s\" occurs %d time(s) in %s\n", word, count,
filename);
    }
    else
    {
        printf("The word \"%s\" was not found in %s\n", word, filename);
    }

    return 0;
}
```

Output:

```
PS C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C> ./test
Enter filename: students.txt
Enter word to search: C
The word "C" was not found in students.txt
PS C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C>
```

Question 3: Dynamic Array Using malloc()

```
#include <stdio.h>
#include <stdlib.h>

int main()
```

```c
{
    int *arr;
    int n, i;
    int max, min;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    arr = (int *)malloc(n * sizeof(int));
    if(arr == NULL)
    {
        printf("Memory allocation failed!\n");
        return 1;
    }

    printf("Enter %d integers: ", n);
    for(i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("You entered: ");
    for(i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");

    max = arr[0];
    min = arr[0];
    for(i = 1; i < n; i++)
    {
        if(arr[i] > max)
        {
            max = arr[i];
        }
        if(arr[i] < min)
        {
            min = arr[i];
        }
    }
```

```c
    printf("Largest element = %d\n", max);
    printf("Smallest element = %d\n", min);

    free(arr);

    return 0;
}
```

Output:

```
PS C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C> gcc test.c -o test
PS C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C> ./test
Enter number of elements: 5
Enter 5 integers: 1 2 3 6 5
You entered: 1 2 3 6 5
Largest element = 6
Smallest element = 1
PS C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C>
```

Question 4: Student Marks Using calloc()

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    float *marks;
    int n, i;
    float sum = 0, average;

    printf("Enter number of students: ");
    scanf("%d", &n);

    marks = (float *)calloc(n, sizeof(float));
    if(marks == NULL)
    {
        printf("Memory allocation failed!\n");
        return 1;
    }

    printf("Enter marks of %d students: ", n);
```

```c
    for(i = 0; i < n; i++)
    {
        scanf("%f", &marks[i]);
        sum = sum + marks[i];
    }

    average = sum / n;

    printf("Marks entered: ");
    for(i = 0; i < n; i++)
    {
        printf("%.2f ", marks[i]);
    }

    printf("\nAverage marks = %.2f\n", average);

    free(marks);

    return 0;
}
```

Output:

```
PS C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C> gcc test.c -o test
PS C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C> ./test
Enter number of students: 2
Enter marks of 2 students: 45 65
Marks entered: 45.00 65.00
Average marks = 55.00
PS C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C>
```

Question 5: Even and Odd Count

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *arr;
    int n, i;
    int even = 0, odd = 0;
```

```
    printf("Enter number of integers: ");
    scanf("%d", &n);

    arr = (int *)calloc(n, sizeof(int));
    if(arr == NULL)
    {
        printf("Memory allocation failed!\n");
        return 1;
    }

    printf("Enter %d integers: ", n);
    for(i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);

        if(arr[i] % 2 == 0)
        {
            even++;
        }
        else
        {
            odd++;
        }
    }

    printf("Numbers entered: ");
    for(i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }

    printf("\nEven numbers = %d\n", even);
    printf("Odd numbers = %d\n", odd);

    free(arr);

    return 0;
}
```

Output:

```
PS C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C> gcc test.c -o test
PS C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C> ./test
Enter number of integers: 4
Enter 4 integers: 9 8 7 4
Numbers entered: 9 8 7 4
Even numbers = 2
Odd numbers = 2
PS C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C>
```

Question 6: Expand an Array

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *arr;
    int n, m, i;

    printf("Enter initial number of elements: ");
    scanf("%d", &n);

    arr = (int *)malloc(n * sizeof(int));
    if(arr == NULL)
    {
        printf("Memory allocation failed!\n");
        return 1;
    }

    printf("Enter %d integers: ", n);
    for(i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("Enter how many additional elements to add: ");
    scanf("%d", &m);

    arr = (int *)realloc(arr, (n + m) * sizeof(int));
    if(arr == NULL)
    {
```

```
        printf("Memory reallocation failed!\n");
        return 1;
    }

    printf("Enter %d more integers: ", m);
    for(i = n; i < n + m; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("Complete array: ");
    for(i = 0; i < n + m; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");

    free(arr);

    return 0;
}
```

Output:

```
PS C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C> gcc test.c -o test
PS C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C> ./test
Enter initial number of elements: 3
Enter 3 integers: 8 5 2
Enter how many additional elements to add: 2
Enter 2 more integers: 9 6
Complete array: 8 5 2 9 6
PS C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C>
```

Question 7: Student List Update

```
#include <stdio.h>
#include <stdlib.h>

struct Student
{
    int rollNo;
    char name[50];
```

```c
};

int main()
{
    struct Student *s;
    int n = 3, extra = 2, i;

    s = (struct Student *)malloc(n * sizeof(struct Student));
    if(s == NULL)
    {
        printf("Memory allocation failed!\n");
        return 1;
    }

    printf("Enter details of %d students:\n", n);
    for(i = 0; i < n; i++)
    {
        printf("Roll No: ");
        scanf("%d", &s[i].rollNo);

        printf("Name: ");
        scanf("%s", s[i].name);
    }

    s = (struct Student *)realloc(s, (n + extra) * sizeof(struct Student));
    if(s == NULL)
    {
        printf("Memory reallocation failed!\n");
        return 1;
    }

    printf("\nEnter details of %d more students:\n", extra);
    for(i = n; i < n + extra; i++)
    {
        printf("Roll No: ");
        scanf("%d", &s[i].rollNo);

        printf("Name: ");
        scanf("%s", s[i].name);
    }
```

```c
    printf("\n--- All Student Records ---\n");
    for(i = 0; i < n + extra; i++)
    {
        printf("Roll No: %d, Name: %s\n", s[i].rollNo, s[i].name);
    }

    free(s);

    return 0;
}
```

Output:

```
PS C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C> gcc test.c -o test
PS C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C> ./test
Enter details of 3 students:
Roll No: 1
Name: Amit
Roll No: 2
Name: Akshansh
Roll No: 3
Name: Aryan

Enter details of 2 more students:
Roll No: 4
Name: Golu
Roll No: 5
Name: Shani

--- All Student Records ---
Roll No: 1, Name: Amit
Roll No: 2, Name: Akshansh
Roll No: 3, Name: Aryan
Roll No: 4, Name: Golu
Roll No: 5, Name: Shani
PS C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C>
```

Question 8: Employee Information Using enum and union

```c
#include <stdio.h>

enum EmployeeType
{
    Manager,
    Engineer,
    Intern
};
```

```c
union EmployeeData
{
    float salary;
    float stipend;
};

int main()
{
    enum EmployeeType type;
    union EmployeeData data;
    int choice;

    printf("Select employee type:\n");
    printf("0. Manager\n");
    printf("1. Engineer\n");
    printf("2. Intern\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    type = (enum EmployeeType)choice;

    if(type == Intern)
    {
        printf("Enter stipend: ");
        scanf("%f", &data.stipend);
    }
    else
    {
        printf("Enter monthly salary: ");
        scanf("%f", &data.salary);
    }

    printf("\nEmployee Type: ");
    switch(type)
    {
        case Manager:
            printf("Manager\n");
            printf("Monthly Salary: %.2f\n", data.salary);
            break;
```

```c
        case Engineer:
            printf("Engineer\n");
            printf("Monthly Salary: %.2f\n", data.salary);
            break;

        case Intern:
            printf("Intern\n");
            printf("Stipend: %.2f\n", data.stipend);
            break;

        default:
            printf("Invalid type\n");
    }

    return 0;
}
```

Output:

```
PS C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C> gcc test.c -o test
PS C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C> ./test
Select employee type:
0. Manager
1. Engineer
2. Intern
Enter choice: 2
Enter stipend: 20000

Employee Type: Intern
Stipend: 20000.00
PS C:\Users\STUDENT\Desktop\Amit\C-PROGRAMMING-COURSE\C>
```

MCQ Questions: Answers

Q	Answer	Explanation
1	B) "w+"	"w+" opens a file for both reading and writing; it creates the file if it doesn't exist, and truncates (empties) it if it does.
2	B) ABCHello	"a" (append) mode opens the file for writing at the end without erasing existing content, so "Hello" is added after the existing "ABC".
3	B) int *p = malloc(20 * sizeof(int));	malloc() takes a size in bytes; multiplying the element count by sizeof(int) correctly reserves space for 20 integers regardless of int's size on the platform.
4	B) It initializes allocated memory to zero.	Unlike malloc(), which leaves allocated memory uninitialized, calloc() clears every byte of the allocated block to zero before returning it.

Q	Answer	Explanation
5	B) It may move the allocated block to a new location.	If the current block cannot be resized in place, realloc() allocates a new block elsewhere, copies the existing data over, and frees the old block - so the returned pointer can differ from the original.
6	C) Original memory remains allocated, but assigning directly to ptr may cause a memory leak.	If realloc() fails it returns NULL without freeing the original block; writing ptr = realloc(ptr, new_size) directly overwrites ptr with NULL, losing the only reference to the still-allocated original memory.
7	B) ProXYZmming	"Programming" is written first; fseek(fp, 3, SEEK_SET) moves the file position to offset 3, and "XYZ" overwrites the 3 characters starting there ('g','r','a'), leaving "Pro" + "XYZ" + "mming".
8	C) 5	MON is explicitly set to 2, so the following constants increment automatically: TUE = 3, WED = 4, THU = 5.

Explain

1. What is the difference between fputc() and fgetc()?

fputc(int ch, FILE *stream) writes a single character to a file and returns the character written, or EOF if the write fails. fgetc(FILE *stream) reads a single character from a file and returns it as an int, or returns EOF when the end of the file is reached or an error occurs. fputc() is used for character output, fgetc() for character input; both operate one character at a time and automatically move the file's internal position indicator forward.

2. Explain the use of fputs() and fprintf().

fputs(const char *str, FILE *stream) writes a plain string to a file exactly as given - it does not interpret format specifiers and does not add a trailing newline automatically. fprintf(FILE *stream, const char *format, ...) writes formatted output to a file, working just like printf() but directed at a file stream, so it can combine literal text with variables of different types using format specifiers such as %d, %s, and %f. fputs() is used when a ready-made string needs to be written as-is, while fprintf() is used when the output has to be built from a format string and one or more values.

3. Explain the syntax of malloc().

void *malloc(size_t size);

malloc() takes the number of bytes to allocate and returns a void pointer to the first byte of that memory block. The returned pointer is normally cast to the required type, for example: int *p = (int *)malloc(n * sizeof(int)); which reserves enough space for n integers.

4. What does malloc() return on failure?

If malloc() cannot satisfy the requested allocation (for instance, if the system has run out of memory), it returns NULL instead of a valid address. A program should always check the returned pointer against NULL before using it, since reading or writing through a NULL pointer causes undefined behavior, typically crashing the program.

5. A programmer forgets to call free() after using malloc(). What problems may arise if this happens repeatedly?

Every block that is allocated with malloc() but never released with free() stays reserved for as long as the program keeps running - this is called a memory leak. If this happens repeatedly, for example inside a loop or inside a function that gets called many times, the amount of memory available to the program keeps shrinking over time. Eventually this can cause further allocation calls to fail, degrade performance due to excessive memory use, or crash the program (or, in severe cases, affect the

whole system). The problem is especially serious in long-running programs such as servers, which may need to keep running for days or months without restarting.

6. How is calloc() different from malloc()?

malloc(size) allocates a single block of the given number of bytes but leaves its contents uninitialized, so the memory holds garbage values until it's explicitly written to. calloc(n, size) allocates memory for n elements of size bytes each and additionally initializes every byte of that memory to zero before returning it. calloc() also takes two arguments - element count and element size - rather than malloc()'s single byte-count argument, which makes it more convenient and less error-prone when allocating arrays.

7. Explain the syntax and working of calloc() with an example.

void *calloc(size_t n, size_t size);

calloc() allocates memory large enough for n elements of size bytes each, clears the entire block to zero, and returns a pointer to the start of the block (or NULL if the allocation fails). For example:

```
int *arr = (int *)calloc(5, sizeof(int));
/* allocates space for 5 integers, and every one of them starts out as 0 */
```

8. Can realloc() reduce the size of allocated memory? Explain.

Yes. realloc(ptr, new_size) can be used to both grow and shrink a previously allocated block. If new_size is smaller than the block's current size, realloc() typically resizes the block in place and keeps the existing data up to the new size, discarding whatever no longer fits. If new_size is larger, the block may need to be moved to a new location, with the existing content copied over automatically. If new_size is passed as 0, the behavior is implementation-defined, but on many systems it behaves the same as calling free(ptr).

9. Explain the syntax and advantages of enum with an example.

enum EnumName { CONST1, CONST2, CONST3, ... };

By default, the first name in the list is given the value 0 and each following name is automatically one more than the previous one, though any constant can also be assigned an explicit value, as in enum Day {MON = 2, TUE, WED, THU}; from the MCQ section above, where TUE, WED, and THU automatically become 3, 4, and 5. The advantages of enum are: it gives meaningful, readable names to a set of related integer constants instead of using plain numbers ("magic numbers") scattered through the code; it groups related constants together under a single type, as with EmployeeType in Question 8 above; and it makes the code easier to read and maintain compared to using #define or raw integer literals for the same purpose.

10. In what situations would you prefer using a union instead of a structure? Explain with an example.

A union is preferable when only one of several possible members needs to be stored at any given moment, and memory efficiency matters - all members of a union share the same block of memory, so a union's total size is only as large as its biggest member, unlike a structure, which reserves separate space for every member at once. Question 8 above is a direct example: an employee is a Manager, Engineer, or Intern, and at any given time only one value needs to be stored for them - either a salary or a stipend, never both together. Using a union for EmployeeData means salary and stipend share the same memory location, so the union only needs to be as large as a single float, whereas a structure with both fields would reserve space for both, wasting memory that would never be used for a given employee.